Deep Avance mini-project

Assignment Report

Eng. Robert Bejan

I. Introduction

This mini-project provides a framework for the geometric characterization and statistical classification of digital objects. It begins by extracting connected components from binary images and converting their discrete boundaries into polygonal approximations using a greedy segmentation algorithm. Essential shape descriptors (area, perimeter, and circularity) are computed from these vectorized contours to quantify morphological differences. By first analyzing homogenous datasets, the system establishes statistical features (mean and standard deviation) for known categories; it then utilizes these profiles to classify objects within mixed populations, assigning a class based on the minimal statistical deviation (Z-score) from the learned reference characteristics.

II. Application walkthrough

The methodology begins by converting binary input images into digital sets and establishing a digital topology using (4,8)-adjacency. This setting ensures that foreground objects are 4-connected while the background is 8-connected, allowing for the correct extraction of individual connected components. A filtering step is immediately applied to discard any components that touch the image borders, ensuring that only complete objects are analyzed (implementation in the code below).

```
// Filter boundary grains
std::vector<ObjectType> completeObjects;
Domain domain = image.domain();
Point pMin = domain.lowerBound();
Point pMax = domain.upperBound();
for (const auto& obj : objects) {
    bool touches = false;
    for (auto const& p : obj.pointSet()) {
        if (p[0] == pMin[0] || p[0] == pMax[0] || p[1] == pMin[1] || p[1] == pMax[1]) {
            touches = true; break;
        }
    }
    if (!touches) completeObjects.push_back(obj);
}
std::cout << "  > Grains found: " << completeObjects.size() << endl;
```

To extract the boundary of a digital object, the code transitions from a simple set of pixels to a topological representation using a Khalimsky Space **(**KSpace**)**. A KSpace is a cellular grid that explicitly represents not just the pixels (2-cells), but also the edges between them (1-cells) and the vertices (0-cells). The code initializes this space with a domain slightly larger than the object itself to ensure the boundary is fully contained.

Once the topological space is defined, the algorithm identifies a single starting boundary element (called a "bel" - boundary element) or surfel (surface element), which represents a 1-cell interface between an object pixel and a background pixel. Using Surfaces<KSpace>::findABel, the code locates this initial interface. From there, the implementation below utilizes Surfaces<KSpace>::track2DBoundary to trace the entire contour of the connected component. This

tracking follows the adjacency of 1-cells, walking around the perimeter of the object until it returns to the start, resulting in a closed sequence of boundary segments (or a Curve).

Implementation of `getBoundary` function:

```
template<class T>
Curve getBoundary(T & object)
{
   const DigitalSet& set = object.pointSet();
   if (set.empty()) return Z2i::Curve();

   Point lower = set.domain().upperBound();
   Point upper = set.domain().lowerBound();

   for (const auto& p : set) {
       lower = lower.inf(p);
       upper = upper.sup(p);
   }

   KSpace kSpace;
   kSpace.init(lower - Point(2,2), upper + Point(2,2), true);

   Point pIn  = *set.begin();
   Point pOut = kSpace.lowerBound();
   SCell s = Surfaces<KSpace>::findABel(kSpace, set, pIn, pOut);

   std::vector<SCell> boundarySurfels;
   SurfelAdjacency<2> sAdj(true);
   Surfaces<KSpace>::track2DBoundary(boundarySurfels, kSpace, sAdj, set, s);

   Z2i::Curve boundaryCurve;
   boundaryCurve.initFromSCellsVector(boundarySurfels);

   return boundaryCurve;
}
```

Implementation of the visualization:

```
          if (!completeObjects.empty()) {
              Z2i::Curve c = getBoundary(completeObjects[0]);
              Z2i::Curve::PointsRange range = c.getPointsRange();
              Decomposition4 theDecomposition( range.c(), range.c(), DSS4() );

              Board2D aBoard;
              aBoard << completeObjects[0];
              for (auto it = theDecomposition.begin(); it != theDecomposition.end(); ++it){
                  aBoard << SetMode( "ArithmeticalDSS", "Points" )
                         << it->primitive()
                         << SetMode( "ArithmeticalDSS", "BoundingBox" )
                         << CustomStyle("ArithmeticalDSS/BoundingBox", new CustomPenColor(Color::Red))
                         << it->primitive();
              }
              std::string pdfName = "Analyzed_" + grainType + ".pdf";
              aBoard.saveCairo(pdfName.c_str(), Board2D::CairoPDF);
          }
```

Next, the application converts the "staircase-like" pixel boundary into a clean geometric polygon. The code first takes the extracted boundary curve and retrieves its sequence of points using getPointsRange(). It then passes these points to the **Greedy Segmentation** algorithm (Decomposition4 in the code). This algorithm iterates through the boundary pixels and groups them into the longest possible straight segments that are mathematically valid as "Digital Straight Segments" (DSS). The result is a decomposition of the pixel shape into a simplified sequence of straight lines (a polygon), which allows for accurate geometric calculations like area and perimeter.

Implementation of the digital object boundary:

```
if (!completeObjects.empty()) {
    Z2i::Curve c = getBoundary(completeObjects[0]);
    Z2i::Curve::PointsRange range = c.getPointsRange();
    Decomposition4 theDecomposition( range.c(), range.c(), DSS4() );

    Board2D aBoard;
    aBoard << completeObjects[0];
    for (auto it = theDecomposition.begin(); it != theDecomposition.end(); ++it){
        aBoard << SetMode( "ArithmeticalDSS", "Points" )
        << it->primitive()
        << SetMode( "ArithmeticalDSS", "BoundingBox" )
        << CustomStyle("ArithmeticalDSS/BoundingBox", new CustomPenColor(Color::Red))
        << it->primitive();
    }
    std::string pdfName = "Analyzed_" + grainType + ".pdf";
    aBoard.saveCairo(pdfName.c_str(), Board2D::CairoPDF);
}
```

This next step focuses on quantifying the size of each grain. While the text suggests two methods (pixel count and polygon area), the provided code implements the Polygon Area method. It calculates the area of the vectorized shape obtained in the previous segmentation step. To do this, the code utilizes the Shoelace Formula (also known as the Surveyor's formula). Instead of counting individual pixels, this mathematical approach uses the coordinates of the polygon's vertices.

The algorithm iterates through the list of vertices, calculating the cross-product for each pair of adjacent points (xi*yi+1−xi+1*yi). These values are summed up to produce a signed area. Finally, the absolute value of this sum is divided by two to get the precise area of the polygon. This metric is generally preferred for shape analysis because it is mathematically consistent with the polygon perimeter calculated from the same vertices.

```
std::vector<Point> vertices;
    for (auto it = theDecomposition.begin(); it != theDecomposition.end(); ++it) {
        if (vertices.empty() || vertices.back() != *it->begin()) {
            vertices.push_back(*it->begin());
        }
    }
double polygonalArea = 0.0;
int n = vertices.size();
for (int j = 0; j < n; j++) {
    int idx = (j + 1) % n;
    polygonalArea += (double)vertices[j][0] * vertices[idx][1];
```

```
        polygonalArea -= (double)vertices[idx][0] * vertices[j][1];
    }
    polygonalArea = std::abs(polygonalArea) / 2.0;
```

This next snip of code calculates the boundary length of the grain. Similar to the area calculation, the text proposes two methods, but the code specifically implements Polygon Perimeter. This method provides an accurate estimation of the true perimeter rather than simply counting pixel edges (1-cells).

To calculate this, the algorithm iterates through the ordered vertices of the polygon generated in the last step. For every pair of consecutive vertices, it calculates the Euclidean distance using the standard formula sqrt((x2−x1)^2+(y2−y1)^2). These individual segment lengths are summed up to produce the total perimeter of the grain. This metric is important because, when combined with area, it allows for the calculation of Circularity, which is the primary feature used for classification in this project.

```
    double polygonalPerimeter = 0.0;
    for (int j = 0; j < n; j++) {
        int idx = (j + 1) % n;
        double dx = vertices[j][0] - vertices[idx][0];
        double dy = vertices[j][1] - vertices[idx][1];
        polygonalPerimeter += std::sqrt(dx*dx + dy*dy);
    }
```

This step synthesizes the previously calculated Area and Perimeter into a single, dimensionless shape descriptor called Circularity. The code implements the standard formula: C=4π*Area/Perimeter^2. This feature is important because it quantifies how close a shape is to a perfect circle versus how elongated it is.

Because the Area and Perimeter were derived from the polygonal reconstruction rather than simple pixel counting, these measurements maintain multigrid convergence. This means the circularity value is a mathematically robust feature, independent of the image resolution or grain size. This makes it the best component for distinguishing between the rounder Japonais rice and the elongated Basmati rice.

```
    double circularity = 0.0;
    if (polygonalPerimeter > 0.0) {
    circularity = (4.0 * M_PI * polygonalArea) / (polygonalPerimeter * polygonalPerimeter);}
```

Finally, to establish a reliable baseline for identification, the system aggregates these values for every grain, applies a filter to discard outliers (broken grains or noise), and computes the statistical mean and standard deviation for each rice type, exporting all raw and summarized data to CSV files for verification.

Statistical details for each grain type:

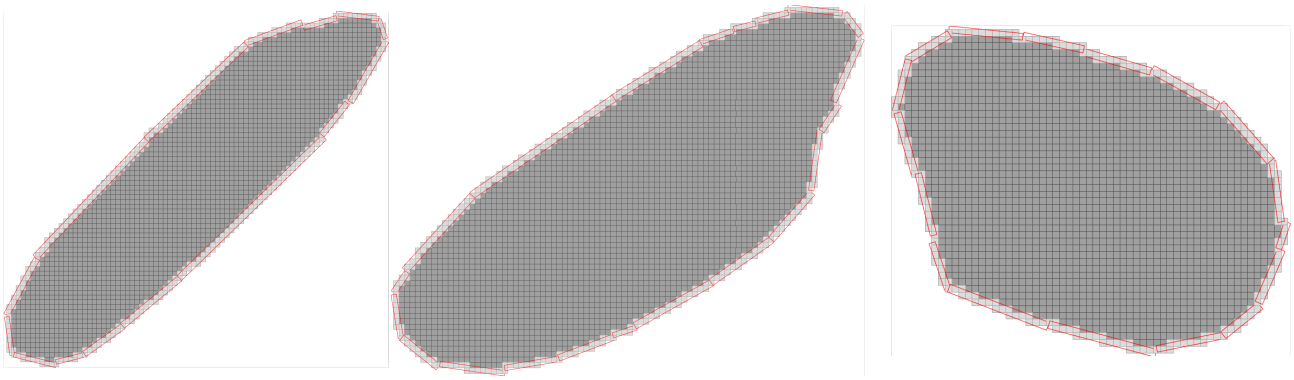
Grain Type	Area Mean	Area Std Dev	Area Min	Area Max	Peri Mean	Peri Std Dev	Peri Min	Peri Max	Circ Mean	Circ Std Dev	Circ Min	Circ Max
Japonais	2048.6	140.365	1726	2363	173.05	5.87446	158.1	186.8	0.8588	0.02100	0.7890	0.9178
Basmati	2287.5	268.422	1516	2976	234.05	17.1473	172.5	274.3	0.5254	0.04109	0.4365	0.6486
Camargue	2787.9	328.438	1749	3368	219.21	13.5373	160.8	262.2	0.7278	0.04411	0.5736	0.8493

To classify the mixed grains, I implemented the statistical approach based on the Circularity profiles established in the previous steps. First, I hard-coded the learned statistics (Mean and Standard Deviation) for Japonais, Basmati, and Camargue rice into data structures. Then, for every grain in the mixed images, the code calculates its circularity and compares it against these three profiles using a Z-Score (which measures how many standard deviations a data point is from the mean). The grain is automatically classified as the rice type for which it has the lowest Z-Score (the smallest statistical distance). Finally, the code displays the number of grains for each class that appear in the binary image.

III. Results

1. The binary image of grains as visualized in the pdf output file (Step 2). To be noticed the absence of grains that are located on the border of the image.

2. The output pdf file of rice grain of all types with the boundary boxes set on the frontier of the actual digital object (in red). This result concludes Step 3 and 4.



3. Below, the console log of the entire application runtime shows the load and the statistical analysis for each of each type of rice grain, along the solutions to the classification problems.

```
Analyzing: Japonais (../RiceGrains/Rice_japonais_seg_bin.pgm) ---
  > Grains found: 138
  > Avg Area: 2048.57 | Avg Circ: 0.858839

Analyzing: Basmati (../RiceGrains/Rice_basmati_seg_bin.pgm) ---
  > Grains found: 124
  > Avg Area: 2287.53 | Avg Circ: 0.525403

Analyzing: Camargue (../RiceGrains/Rice_camargue_seg_bin.pgm) ---
  > Grains found: 112
  > Avg Area: 2787.95 | Avg Circ: 0.727868

Classifying: Rice_mixed2_seg_bin.pgm ---
  > Classification Counts:
    - Basmati: 115
    - Camargue: 20
    - Japonais: 67
  > Visualization saved: Classified_Rice_mixed2_seg_bin.pgm.pdf

Classifying: Rice_mixed3_seg_bin.pgm ---
  > Classification Counts:
    - Basmati: 96
    - Camargue: 54
    - Japonais: 28
  > Visualization saved: Classified_Rice_mixed3_seg_bin.pgm.pdf

Tasks complete.
```